PREDICTING FALCON 9 FIRST-STAGE LANDING SUCCESS

IBM Applied Data Science Capstone

Data collection • Wrangling • EDA • SQL • Folium • Plotly Dash • Classification

Student name: _____

Your GitHub repository URL: _____

Transparency: You told me you have not completed the labs. This document therefore uses findings reported in publicly available examples of the same IBM capstone as background, not as results produced by you. Cite and verify sources; replace the repository field with your own work. Do not represent another person's repository or screenshots as yours.

1. Executive Summary

The project asks whether public Falcon 9 launch characteristics can help predict a successful first-stage landing. The course workflow combines API collection, web scraping, data cleaning, exploratory analysis, SQL, geospatial mapping, an interactive dashboard, and supervised classification. Reusability matters because a recoverable first stage can reduce launch costs.

Reported findings in a public example project: launch success improves over time; launch site and orbit are associated with different observed success rates; and classification models such as Logistic Regression, SVM, Decision Tree, and KNN can be compared. One public implementation reports a Decision Tree test accuracy of 88.9%, while another reports 94.44%; results differ with dataset, split, and tuning, so these numbers are examples, not interchangeable guarantees. [1][2]

- Business question: estimate first-stage landing outcome from available mission features.
- Methods: data collection, wrangling, EDA, SQL, mapping, dashboarding, and binary classification.
- Decision support: identify patterns and compare models; avoid causal claims from observational data.

2. Introduction and Problem Statement

Context. SpaceX's partially reusable Falcon 9 system makes first-stage recovery a meaningful operational question. A competitor evaluating launch pricing would benefit from understanding which launch characteristics correlate with recovery outcomes. [1][2]

Research question: Given information such as payload mass, launch site, orbit, and flight number, can a model estimate whether the first stage will land successfully?

- Target: binary landing class (0 = unsuccessful, 1 = successful), following the course lab convention.
- Candidate features: payload mass, launch site, orbit, flight number, booster information where available.
- Evaluation: held-out test data, confusion matrix, accuracy, F1-score, and Jaccard score.

3. Data Collection — SpaceX API

The course workflow uses SpaceX's REST API to retrieve launch records, normalize JSON into tabular form, and filter the data to Falcon 9 launches. Typical fields include launch date, launch site, payload, orbit, booster/flight details, and landing outcome. [1][2]

Workflow: API request → parse JSON → normalize/select fields → inspect missing values → save the dataset and document the endpoint.

- Record the exact endpoint and retrieval date from your lab.
- Preserve the raw response where practical and document field transformations.
- Validate unique launch identifiers, date parsing, and numeric payload fields.
- Add the notebook/script to your own repository and put that URL in the final submission.

4. Data Collection — Web Scraping

The course also scrapes a launch-history table (commonly from Wikipedia) with Requests and BeautifulSoup. The table is parsed into records and converted into a Pandas DataFrame. This source complements API-derived fields with mission/launch table information. [1][2]

Workflow: request page → parse HTML → extract table headers and rows → normalize values → validate records.

- Document the exact page URL used in your lab.
- Check for repeated headers, merged cells, missing values, and date-format inconsistencies.
- Compare shared fields with the API dataset where possible.
- Include scraper code and a short description of the extracted fields.

5. Data Wrangling Methodology

Task	Method	Validation evidence
Filtering	Keep Falcon 9 launch records relevant to the prediction question.	Row count before/after filter
Missing data	Inspect missingness; impute or exclude with documented rationale.	Missing-value summary
Target label	Map the course-defined landing outcomes to 0/1.	Class counts
Types/features	Parse dates and numeric payload/flight fields; encode categories.	Data types and feature list
Model pipeline	Fit scaling/encoding on training data only to prevent leakage.	Train/test split and pipeline

Insert actual row counts and missing-value decisions from your course notebook. The exact counts are dataset-version dependent.

6. EDA with Visualization

The visualization lab explores landing outcomes against payload, launch site, flight number, orbit, and time. Typical plots include scatter plots, bar charts, and yearly trends. The results should be generated from the course dataset rather than copied without verification. [1][2]

- Payload mass vs. landing outcome: compare successful and unsuccessful outcomes across payload mass.
- Launch site vs. outcome: compare counts and proportions by site.
- Yearly trend: calculate launch/landing success over time.
- Orbit and booster comparisons: inspect sample sizes before interpreting apparent 100% success rates.

Interpretation caution: a high observed rate for a rare orbit may be based on very few launches; report sample size with rates.

7. EDA with SQL

SQL queries summarize launch outcomes and support questions about launch-site frequency, orbit popularity, payload totals, and time trends. The course uses SQLite/SQL syntax in its lab workflow. [1][2]

```
SELECT Launch_Site, COUNT(*) AS total_launches
FROM SPACEXTBL
GROUP BY Launch_Site
ORDER BY total_launches DESC;

SELECT Orbit, COUNT(*) AS launches
FROM SPACEXTBL
GROUP BY Orbit
ORDER BY launches DESC;
```

- Use the exact table and field names from your notebook.
- Add SQL result tables/screenshots for launch sites, payloads, success rates, rankings, and time analysis.
- Check how unknown outcomes are handled in success-rate calculations.

8. Interactive Visual Analytics — Folium and Plotly Dash

Folium map	Plotly Dash dashboard
• Map the launch-site coordinates. • Add markers, labels, and popups. • Use marker colours to distinguish outcomes if records support it. • Analyze proximity to coast/city/roads only when coordinates are available.	• Provide a launch-site dropdown and payload-range slider if implemented. • Pie chart: successful vs. unsuccessful outcomes. • Scatter plot: payload mass vs. landing outcome. • Show source and make filters responsive.

The rubric specifically asks for the Folium map and Plotly Dash app URLs and result slides. These must be tested and documented; this PDF does not claim to include a running app.

9. EDA Visualization Results

Findings reported by public examples of this capstone (verify against your data):

• Landing success is reported to improve with flight experience/time in the analyzed historical window. [1][2]

• KSC LC-39A is reported as having a high landing-success rate in one public analysis; the precise rate depends on the dataset and definition. [1][2]

• Payload mass and orbit show different outcome patterns, but relationships should not be interpreted as causal. [1][2]

Required evidence for your submission: actual scatter plot, bar chart, and yearly trend chart, each with a short interpretation and GitHub URL. The bullets above are secondary-source summaries, not analysis run by you.

10. EDA with SQL — Results

The SQL portion should present compact outputs for these required topics:

- Launch-site rankings and total launch counts.
- Payload summaries and successful-payload ranges.
- Counts of successful and unsuccessful landing outcomes.
- Orbit success rates, including denominators/sample sizes.
- Yearly trends and time-based rankings.

Public project examples report launch-site and orbit differences, but exact values vary across datasets and outcome definitions. Replace this checklist with screenshots or tables from your own SQL results. [1][2]

11. Folium Map — Launch Sites and Proximity

Folium makes the launch-site coordinates explorable on an interactive map. The map slide should show each site marker and a popup with its name and useful metadata. Proximity analysis in the course asks about nearby features such as coastlines, cities, roads, or rail lines where coordinates are available. [1][2]

Interpretation reported in public examples: launch sites are near coasts; some are at relatively low latitudes, which can be operationally advantageous for launches. Confirm map markers and distance measurements against the actual lab output. [1][2]

Insert a screenshot of your rendered Folium map and list the source of coordinates.

12. Plotly Dash — Interactive Results

The dashboard rubric requires a success pie chart and payload-versus-outcome scatter plot. Typical controls include a dropdown for launch site and a range slider for payload mass. These controls should update the displayed visualizations. [1][2]

- Pie chart: success/failure split for all sites or a selected site.
- Scatter chart: payload mass vs. outcome, optionally coloured by class/booster version.
- Show chart labels, legends, units, and meaningful filters.
- Include a screenshot of the app running and the actual source-code URL.

13. Predictive Analysis — Model Comparison

The course compares four classifiers: Logistic Regression, Support Vector Machine, Decision Tree, and K-Nearest Neighbors. Use a held-out test set and compare metrics such as accuracy, F1-score, Jaccard score, and confusion matrix. Hyperparameter tuning should use training data/cross-validation only. [1][2]

Model	Role in comparison	What to report
Logistic Regression	Interpretable baseline	Accuracy, F1, Jaccard, confusion matrix
SVM	Margin-based classifier	Accuracy, F1, Jaccard, confusion matrix
Decision Tree	Rule-based classifier	Accuracy, F1, Jaccard, confusion matrix
KNN	Distance-based classifier	Accuracy, F1, Jaccard, confusion matrix

Public examples report a Decision Tree as the top model, with reported accuracy differing between implementations (88.9% in [2]; 94.44% in [1]). Do not claim either score as your own unless reproduced using your dataset and split.

14. Conclusion, Limitations, and Recommendations

This capstone combines data engineering, exploratory analysis, geospatial visualization, interactive analytics, and supervised learning to study first-stage landing outcomes. Public examples indicate that landing outcomes vary with launch site, orbit, payload, and launch history; however, the strength of each relationship must be assessed on the specific dataset used. [1][2]

- Limitations: historical data coverage, missing fields, class imbalance, and small sample sizes in some orbit groups.
- Avoid data leakage; use consistent outcome definitions and disclose the train/test split.
- Do not treat correlations as causes or assume a high rate based on very few launches is robust.
- Recommendations: verify the pipeline, add actual charts/maps/dashboard screenshots, and report real model metrics.

15. References and Submission Checklist

[1] RedBlueSapper, *Applied-Data-Science-Capstone*, public GitHub repository with notebooks, datasets, dashboard code, and a project summary: github.com/RedBlueSapper/Applied-Data-Science-Capstone

[2] Daniel Eduardo López, *IBM-SpaceX*, public GitHub repository describing the capstone methodology, datasets, and reported results: github.com/DanielEduardoLopez/IBM-SpaceX

- Replace the student-name and own-repository fields on the cover.
- Attach actual API, scraping, wrangling, EDA, SQL, Folium, Dash, and model outputs from your own course labs.
- Ensure the PDF includes the completed presentation slides and actual evidence requested in the rubric.
- Do not submit another student's repository or results as your own.

This is an improved, source-attributed emergency draft—not a substitute for completing the labs. Its secondary-source findings are cited and should be verified before being presented as your own results.